Week 2: 10/2/2026

Design and develop an interactive **Streamlit-based web application** that demonstrates the use of essential Streamlit UI components and basic data handling features. The application should provide a simple interface for displaying personal information, taking user inputs, showing/hiding content dynamically, selecting options, maintaining a counter, displaying tabular data, uploading external CSV files, displaying images, and navigating through a sidebar menu.

The developed application must include the following functionalities:

1. Display developer details such as **Name, Role, and Skills**.
2. Accept user input (**Name and Age**) and display the entered data using a **button click**.
3. Use a **checkbox** to show or hide a text message dynamically.
4. Provide a **selectbox** to choose a programming language and display the selected option.
5. Implement a **counter system** using a button that increases the count value on each click.
6. Display a predefined **DataFrame** using Streamlit.
7. Allow the user to **upload a CSV file** and display its contents in the app.
8. Display an **image** using Streamlit.
9. Create a **sidebar menu** with course options such as *Data Science, Full Stack Java, Full Stack Python, and Dot Net*, and display the selected course.
10. Show a **success message** when a button is clicked.

Required Python Libraries

- Streamlit
- Pandas

Installation:

```
pip install streamlit pandas
```

File Structure

```
streamlit_assignment/
│
├── app.py
├── requirements.txt
├── sample.csv
└── image.jpg
```

File Description

- **app.py**
  Main Streamlit application code.
- **requirements.txt**
  Contains required libraries to run the project.

- **sample.csv**
  Sample CSV file for testing the upload feature.
- **image.jpg**
  Any image file to test the image display feature.

1) Requirements (Installations)

Step 1: Install Python

Make sure Python 3.9+ is installed.

Check: `python --version`

Step 2: Create a Project Folder

```
streamlit_assignment/
```

Open this folder in **VS Code**.

Step 3: Create Virtual Environment

In VS Code terminal:

```
python -m venv venv
```

Activate it: **Windows**

```
venv\Scripts\activate
```

Step 4: Install Streamlit + Pandas

```
pip install streamlit pandas
```

`sample.csv`:

```
Name,Age,Role
Hemanth,21,Student
Raj,22,Intern
Anu,23,Developer
```

2. (app.py)

```
import streamlit as st
import pandas as pd

# ------------------------------
# Page Title
# ------------------------------
st.title("Streamlit Hands-On Assignment ")
```

```python
# -------------------------------
# 1) Display Name, Role, Skills
# -------------------------------
st.header("1) Display Name, Role, Skills")

st.write(" Name: John Doe")
st.write(" Role: Computer Science Student")
st.write(" Skills: Python, SQL, DSA, Java, Streamlit")

st.divider()

# -------------------------------
# 2) Take user input (Name, Age) and display using button
# -------------------------------
st.header("2) User Input with Button")

name = st.text_input("Enter your name:")
age = st.number_input("Enter your age:", min_value=1, max_value=100)

if st.button("Submit Details"):
    st.success(f"Hello {name}! You are {age} years old.")

st.divider()

# -------------------------------
# 3) Checkbox to show/hide text
# -------------------------------
st.header("3) Checkbox Show/Hide Text")

show_text = st.checkbox("Show text")

if show_text:
    st.write(" This text is visible because checkbox is selected!")
else:
    st.write(" Text is hidden. Select checkbox to show it.")

st.divider()

# -------------------------------
# 4) Selectbox to choose programming language
# -------------------------------
st.header("4) Selectbox - Choose a Programming Language")

language = st.selectbox(
    "Select a language:",
    ["Python", "Java", "JavaScript", "C++", "C#"]
)
```

```python
st.write(f" You selected: {language}")

st.divider()

# ------------------------------
# 5) Simple Counter using button
# ------------------------------
st.header("5) Counter using Button")

if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Increase Counter"):
    st.session_state.count += 1

st.write(" Counter Value:", st.session_state.count)

st.divider()

# ------------------------------
# 6) Display a DataFrame using Streamlit
# ------------------------------
st.header("6) Display a DataFrame")

data = {
    "Name": ["H", "Raj", "Anu"],
    "Marks": [90, 85, 88],
    "Passed": [True, True, True]
}

df = pd.DataFrame(data)
st.dataframe(df)

st.divider()

# ------------------------------
# 7) Upload CSV and display contents
# ------------------------------
st.header("7) Upload CSV and Display Contents")

uploaded_file = st.file_uploader("Upload a CSV file", type=["csv"])

if uploaded_file is not None:
    df_uploaded = pd.read_csv(uploaded_file)
    st.success("CSV Uploaded Successfully!")
    st.dataframe(df_uploaded)

st.divider()
```

```python
# ------------------------------
# 8) Display an Image
# ------------------------------
st.header("8) Display an Image")

st.image("image.jpg", caption="Displayed Image using Streamlit",
use_container_width=True)

st.divider()

# ------------------------------
# 9) Sidebar menu with courses
# ------------------------------
st.sidebar.title(" Course Menu")

course = st.sidebar.selectbox(
    "Choose a course:",
    ["Data Science", "Full Stack Java", "Full Stack Python", "Dot Net"]
)

st.sidebar.success(f"You selected: {course}")

st.header("9) Sidebar Selection Output")
st.write(f" Selected Course: {course}")

st.divider()

# ------------------------------
# 10) Success message when button clicked
# ------------------------------
st.header("10) Success Message on Button Click")

if st.button("Click Me for Success Message"):
    st.success(" Button clicked successfully! Task completed!")
```

3) Run the Streamlit App

In VS Code terminal (inside project folder):

streamlit run app.py

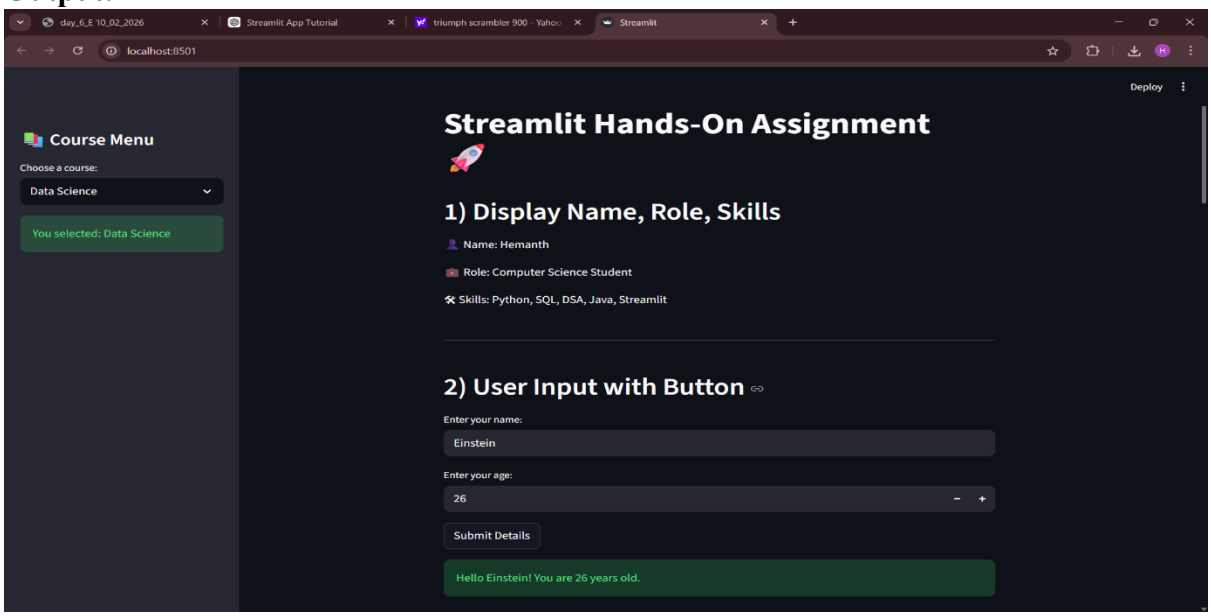
It will open in browser like:

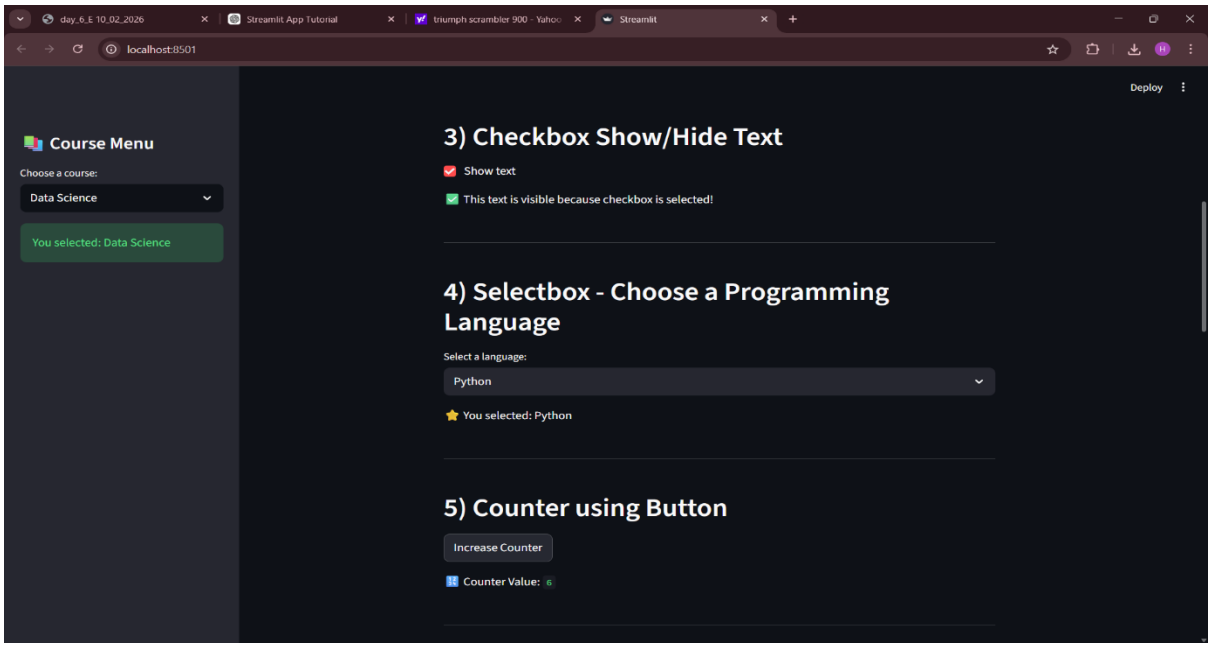
http://localhost:8501

4) Output Features

✓ Display name, role, skills
✓ User input + button display
✓ Checkbox show/hide
✓ Selectbox language selection
✓ Counter using button + session state
✓ Display dataframe
✓ Upload CSV and show it
✓ Display image
✓ Sidebar menu courses
✓ Success message on button click

Output:





Deploy

Course Menu

Choose a course:

Data Science

You selected: Data Science

6) Display a DataFrame

	Name	Marks	Passed
0	H	90	✓
1	Raj	85	✓
2	Anu	88	✓

7) Upload CSV and Display Contents

Upload a CSV file

Drag and drop file here

Limit 200MB per file • CSV

Browse files

day_6_E 10_02_2026 | Streamlit App Tutorial | triumph scrambler 900 - Yahoo | Streamlit

localhost:8501

Deploy

Course Menu

Choose a course:

Data Science

You selected: Data Science

8) Display an Image

9) Sidebar Selection Output

📌 Selected Course: Data Science

10) Success Message on Button Click

Click Me for Success Message

🎉 Button clicked successfully! Task completed!

Task 1: Develop a Streamlit web application that allows the user to upload an employee CSV file and display the employee records. The application must filter and display only those employees whose salary is greater than 50,000. Additionally, the app should include a role selection feature using a dropdown, and based on the selected role, different content should be displayed dynamically.

```
import streamlit as st

import pandas as pd

st.title("Employee Salary Filter App")

st.header("Upload Employee CSV File")

uploaded_file = st.file_uploader("Upload CSV", type=["csv"])

if uploaded_file is not None:

    df = pd.read_csv(uploaded_file)

    st.subheader("Full Employee Data")

    st.dataframe(df)

    st.subheader("Employees with Salary > 50,000")

    if "Salary" in df.columns:

        filtered_df = df[df["Salary"] > 50000]

        st.dataframe(filtered_df)

    else:

        st.error("CSV must contain a 'Salary' column.")

st.divider()

st.header("Role Based Content Display")

role = st.selectbox("Select your role:", ["Student", "Intern", "Employee", "Admin"])

if role == "Student":

    st.write("Welcome Student! You can explore employee data and learn filtering concepts.")

elif role == "Intern":

    st.write("Welcome Intern! You can analyze employee salary data and generate insights.")

elif role == "Employee":

    st.write("Welcome Employee! You can check salary-based employee records.")

elif role == "Admin":

    st.write("Welcome Admin! You have access to all employee records and filtering options.")
```

Week 2: 11/2/2026

1) Problem Statement

Develop a Streamlit-based Inventory Management System using Pandas that allows users to manage product inventory efficiently. The application should support adding new products, removing existing products, searching products, updating stock, viewing the full inventory, and calculating the total inventory value. The system must store and update inventory data in a CSV file so that changes remain saved even after restarting the app.

---

2) Requirements

Software Requirements

- Python 3.8 or above
- VS Code
- Streamlit
- Pandas

Install Required Libraries

```
pip install streamlit pandas
```

---

3) File Structure

Create a folder like this:

```
inventory_app/
│
├── app.py
├── inventory.csv
└── requirements.txt
```

requirements.txt

```
streamlit
pandas
```

inventory.csv (Create this file)

```
ProductID,ProductName,Category,Price,Stock
101,Keyboard,Electronics,799,10
102,Mouse,Electronics,399,20
103,Notebook,Stationery,50,100
104,Pen,Stationery,10,200
```

---

4) Streamlit Code (app.py)

```
import streamlit as st
import pandas as pd
import os

st.title("Inventory Management System")

FILE_NAME = "inventory.csv"

# -------------------------------
# Load Inventory
# -------------------------------
def load_inventory():
    if os.path.exists(FILE_NAME):
        return pd.read_csv(FILE_NAME)
    else:
        return pd.DataFrame(columns=["ProductID", "ProductName",
"Category", "Price", "Stock"])

# -------------------------------
# Save Inventory
# -------------------------------
def save_inventory(df):
    df.to_csv(FILE_NAME, index=False)

# Load Data
df = load_inventory()

st.sidebar.title("Menu")
option = st.sidebar.selectbox(
    "Select an operation:",
    ["View Inventory", "Add Product", "Remove Product", "Search Product",
"Update Stock", "Total Inventory Value"]
)

# -------------------------------
# VIEW INVENTORY
# -------------------------------
if option == "View Inventory":
    st.header("View Inventory")
    st.dataframe(df)

# -------------------------------
# ADD PRODUCT
# -------------------------------
elif option == "Add Product":
    st.header("Add Product")

    pid = st.number_input("Enter Product ID", min_value=1, step=1)
    pname = st.text_input("Enter Product Name")
    category = st.text_input("Enter Category")
    price = st.number_input("Enter Price", min_value=1.0, step=1.0)
    stock = st.number_input("Enter Stock", min_value=0, step=1)

    if st.button("Add Product"):
        if pid in df["ProductID"].values:
            st.error("Product ID already exists. Use a new ID.")
        elif pname.strip() == "" or category.strip() == "":
            st.error("Product Name and Category cannot be empty.")
        else:
```

```
            new_row = {
                "ProductID": pid,
                "ProductName": pname,
                "Category": category,
                "Price": price,
                "Stock": stock
            }
            df = pd.concat([df, pd.DataFrame([new_row])],
ignore_index=True)
            save_inventory(df)
            st.success("Product added successfully!")

# -------------------------------
# REMOVE PRODUCT
# -------------------------------
elif option == "Remove Product":
    st.header("Remove Product")

    pid = st.number_input("Enter Product ID to remove", min_value=1,
step=1)

    if st.button("Remove Product"):
        if pid not in df["ProductID"].values:
            st.error("Product ID not found.")
        else:
            df = df[df["ProductID"] != pid]
            save_inventory(df)
            st.success("Product removed successfully!")

# -------------------------------
# SEARCH PRODUCT
# -------------------------------
elif option == "Search Product":
    st.header("Search Product")

    search_name = st.text_input("Enter Product Name to search")

    if st.button("Search"):
        result = df[df["ProductName"].str.contains(search_name, case=False,
na=False)]
        if result.empty:
            st.warning("No matching product found.")
        else:
            st.dataframe(result)

# -------------------------------
# UPDATE STOCK
# -------------------------------
elif option == "Update Stock":
    st.header("Update Stock")

    pid = st.number_input("Enter Product ID", min_value=1, step=1)
    new_stock = st.number_input("Enter New Stock Value", min_value=0,
step=1)

    if st.button("Update Stock"):
        if pid not in df["ProductID"].values:
            st.error("Product ID not found.")
        else:
            df.loc[df["ProductID"] == pid, "Stock"] = new_stock
            save_inventory(df)
```

```
            st.success("Stock updated successfully!")

# -------------------------------
# TOTAL INVENTORY VALUE
# -------------------------------
elif option == "Total Inventory Value":
    st.header("Total Inventory Value")

    if df.empty:
        st.warning("Inventory is empty.")
    else:
        df["TotalValue"] = df["Price"] * df["Stock"]
        total_value = df["TotalValue"].sum()

        st.subheader("Inventory with Total Value per Product")
        st.dataframe(df)

        st.subheader("Final Total Inventory Value")
        st.success(f"Total Inventory Value = {total_value}")
```

---

5) How to Run (VS Code)

Step 1: Open folder in VS Code

Open the folder `inventory_app`

Step 2: Install libraries

```
pip install streamlit pandas
```

Step 3: Run Streamlit app

```
streamlit run app.py
```

Output Features

✓ Add product
✓ Remove product
✓ Search product
✓ Update stock
✓ View inventory
✓ Total inventory value
✓ Saves updates in `inventory.csv`

Output:

Inventory Management System

View Inventory

	ProductID	ProductName	Category	Price	Stock
0	101	Keyboard	Electronics	799	10
1	102	Mouse	Electronics	399	20
2	103	Notebook	Stationery	50	100
3	104	Pen	Stationery	10	200

Inventory Management System

Add Product

Enter Product ID

105

Enter Product Name

Router

Enter Category

Electronics

Enter Price

3500.00

Enter Stock

30

Add Product

Product added successfully!

Inventory Management System

Remove Product

Enter Product ID to remove

105

Remove Product

Product removed successfully!

Inventory Management System

Search Product

Enter Product Name to search

Keyboard

Search

	ProductID	ProductName	Category	Price	Stock
0	101	Keyboard	Electronics	799	10

Inventory Management System

Update Stock

Enter Product ID

101

Enter New Stock Value

200

Update Stock

Stock updated successfully!

Inventory Management System

Total Inventory Value

Inventory with Total Value per Product

	ProductID	ProductName	Category	Price	Stock	TotalValue
0	101	Keyboard	Electronics	799	200	159800
1	102	Mouse	Electronics	399	20	7980
2	103	Notebook	Stationery	50	100	5000
3	104	Pen	Stationery	10	200	2000

Final Total Inventory Value

Total Inventory Value = 174780.0

Week 2- 12/2/2026

1) Why is NumPy faster than Python lists?

NumPy arrays store data in **contiguous memory blocks** and use **C-optimized loops**, unlike Python lists which store references to objects.
This reduces overhead and enables faster computation using **vectorized operations**.
Also, NumPy uses **SIMD + BLAS/LAPACK** support for speed.

2) What is broadcasting in NumPy?

Broadcasting allows NumPy to perform operations on arrays of **different shapes** without manually resizing them.
It automatically expands the smaller array to match the bigger one logically.
Example: `[1,2,3] + 5 → [6,7,8]`.

3) Features of NumPy

NumPy provides **n-dimensional arrays**, fast computations, and powerful indexing/slicing.
It supports linear algebra, random number generation, and mathematical functions.
It integrates smoothly with ML and data libraries like Pandas, Scikit-learn, TensorFlow.

4) Advantages of NumPy

NumPy is faster than Python lists for numerical operations due to **vectorization**.
It supports advanced operations like matrix multiplication and broadcasting.
It reduces memory usage using fixed-type arrays.

5) What is vectorization and why is it important?

Vectorization means performing operations on entire arrays without explicit Python loops.
It improves speed because calculations happen in optimized C-level implementations.
This is crucial for ML, where large-scale computations are common.

6) How does NumPy integrate with Machine Learning?

NumPy is used for **data preprocessing**, feature scaling, and matrix computations in ML pipelines.
Most ML libraries accept NumPy arrays as input for training and prediction.
It enables fast operations like dot products used in regression and neural networks.

7) Advantages of NumPy in industrial scenarios

Industries use NumPy for **high-performance computing**, analytics, and ML workflows.
It is reliable for large datasets due to optimized speed and memory efficiency.
Used in finance, healthcare, AI automation, and scientific research.

8) Creating 1D and Multi-dimensional arrays

1D array: `np.array([10,20,30])`
2D array: `np.array([[1,2],[3,4]])`
3D array: `np.array([[[1,2],[3,4]],[[5,6],[7,8]]])`

9) NumPy properties (ndim, shape, size, dtype, ndmin)

- `ndim`: number of dimensions
- `shape`: rows/cols structure
- `size`: total elements
  Example: `np.array([[1,2],[3,4]]).shape → (2,2)`

10) Difference between ndim and ndmin

`ndim` tells the **actual dimensions** of an array after creation.
`ndmin` is used while creating an array to **force minimum dimensions**.
Example: `np.array([1,2], ndmin=2)` creates a 2D array.

11) Indexing and slicing examples (1D and 2D)

1D: `a=[10,20,30,40] → a[1]=20, a[1:3]=[20,30]`
2D: `b=[[1,2],[3,4]] → b[0,1]=2, b[:,0]=[1,3]`
Slicing helps extract subarrays efficiently.

12) Linear algebra impact without NumPy in DS/ML/AI

ML heavily depends on matrix operations like dot product, transpose, and inverses.
Without NumPy, these computations would be slower and harder to implement.
NumPy makes ML scalable and computationally practical.

13) Statistical operations and aggregate functions

NumPy provides `mean()`, `median()`, `std()`, `var()`, `sum()`, `min()`, `max()`.
These help analyze datasets quickly with fast numerical performance.
Example: `np.mean(arr)` returns average.

14) What is type() function?

`type()` returns the datatype/class of a variable in Python.
Example: `type(np.array([1,2])) → <class 'numpy.ndarray'>`
It is used to check object type during debugging.

15) Can we do type casting in NumPy? How?

Yes, NumPy supports type casting using `.astype()`.
Example: `arr.astype(float)` converts integers to float.
This is useful for ML preprocessing and memory optimization.

16) Supported data types in NumPy

NumPy supports `int8, int16, int32, int64, float16, float32, float64.`
It also supports `bool`, `complex`, and `string` types.
These types help control memory and precision.

17) How memory is managed in NumPy?

NumPy arrays store fixed-type values in contiguous memory blocks.
This allows faster access and efficient cache utilization.
Unlike Python lists, NumPy avoids object overhead.

18) Importance of NumPy with use cases

NumPy is essential for ML preprocessing, scientific simulation, and analytics.
Used in image processing, financial modeling, and deep learning computations.
It is the base of Pandas, Scikit-learn, and TensorFlow pipelines.

19) What is random in NumPy?

`numpy.random` is a module used to generate random numbers and samples.
It supports randomness for simulation, ML initialization, and testing.
Example: `np.random.rand(3)` generates random floats.

20) Types of random in NumPy

Common random functions include `rand()`, `randint()`, `randn()`, `choice()`.
Also includes distributions like `normal()`, `uniform()`, `binomial()`.
Used in ML for sampling and weight initialization.

BASICS SECTION

21) What is embedding?

Embedding is a technique to represent data (like words/images) as numerical vectors.
It captures meaning or relationships in vector space.
Used in NLP and deep learning models.

22) What is a matrix?

A matrix is a 2D arrangement of numbers in rows and columns.
It is used to represent linear transformations and datasets.
Example: `[[1,2],[3,4]]`

23) What is an array?

An array is a structured collection of elements stored in indexed order.
NumPy arrays store elements of the same datatype efficiently.
Used for fast mathematical operations.

24) Why is it called scientific computing?

Scientific computing involves solving real-world problems using mathematics and computation.
NumPy supports numerical methods, matrix algebra, and simulations.
Used in physics, ML, AI, engineering, and research.

25) Examples of matrix operations

Addition: `A + B`
Multiplication: `A @ B`
Transpose: `A.T`, Determinant: `np.linalg.det(A)`

26) How to install NumPy?

Install using pip: `pip install numpy`
Then import: `import numpy as np`
Works in Python, Jupyter Notebook, and IDEs.

27) Does NumPy version affect integration with libraries?

Yes, incompatible versions can cause errors with Pandas, TensorFlow, Scikit-learn.
Newer versions may deprecate functions or change behavior.
Using stable compatible versions is best for projects.

28) What is data?

Data is a collection of facts, values, or observations used for analysis.
It can be numeric, text, images, audio, or structured tables.
ML uses data to learn patterns and make predictions.

29) Techniques used in NumPy for faster computations

NumPy uses vectorization, broadcasting, and optimized C-based computation.
It supports BLAS/LAPACK for matrix operations.
It reduces Python loop overhead significantly.

30) What is stats?

Statistics is the study of collecting, analyzing, and interpreting data.
It helps summarize data and make predictions.
Used in ML for evaluation and decision-making.

31) What is probability?

Probability measures the chance of an event occurring (0 to 1).
It forms the base for ML models like Naive Bayes.
Used in uncertainty modeling and predictions.

32) What is regression?

Regression is a supervised learning method used to predict continuous values.
Example: predicting house price based on features.
Types include linear regression and polynomial regression.

33) What are distributions?

A distribution describes how values are spread across a dataset.
Examples: normal, uniform, binomial distributions.
Used in statistics, ML, and random sampling.

Week 2- 13/2/2026

1) What are broadcasting rules?

Broadcasting works when array shapes are compatible: compare from the last axis.
Dimensions must be equal OR one of them must be `1`.
Missing dimensions are treated as `1`.

2) Why is broadcasting important in ML?

ML needs fast operations like scaling, normalization, and weight updates.
Broadcasting avoids loops and makes computations faster and cleaner.
It supports efficient batch processing in deep learning.

3) What is a random number?

A random number is a value generated unpredictably within a range.
In computing, most random values are **pseudo-random** (algorithm-based).
Used in simulations, sampling, and ML initialization.

4) Difference between rand and randint?

`rand()` generates random floats between `0 and 1`.
`randint(a,b)` generates random integers between `a and b-1`.
Example: `rand(3)` → floats, `randint(1,10,3)` → integers.

5) Explain sorting in NumPy

NumPy supports sorting using `np.sort()` and `array.sort()`.
`np.sort()` returns a sorted copy, while `array.sort()` sorts in-place.
You can sort along rows/columns using `axis`.

6) How can we search elements in NumPy?

Use `np.where(condition)` to find indexes matching a condition.
Use boolean masking like `arr[arr>10]` for direct filtering.
Use `np.isin()` to check membership of elements.

7) What is joining? Types in NumPy

Joining means combining multiple arrays into one array.
Types: `concatenate()`, `stack()`, `hstack()`, `vstack()`, `dstack()`.
Used for merging features in ML datasets.

8) Explain shape and reshape

`shape` gives the dimension structure like `(rows, cols)`.
`reshape()` changes the array shape without changing data values.
Example: `(6,) → (2,3)` if total elements match.

9) Difference between copy and view

A **view** shares the same memory, so changes affect the original array.
A **copy** creates new memory, so changes do not affect original.
Use `arr.view()` and `arr.copy()`.

10) Difference between flatten() and ravel()

`flatten()` returns a new flattened copy (extra memory).
`ravel()` returns a flattened view when possible (memory efficient).
Both convert multi-D to 1D.

11) Difference between deep copy and shallow copy?

Shallow copy shares references to the same inner objects (partial sharing).
Deep copy creates completely independent objects in new memory.
In NumPy, `.view()` is like shallow copy, `.copy()` is deep copy.

(Task 1: Sales Data Analysis)

You are given sales data containing **Month, Region, and Sales amount**.
Write a Python program using **NumPy and Pandas** to:

1. Calculate **total sales per region**
2. Calculate **average sales per month**
3. Find the **best performing region** (highest total sales)

Sample Dataset

Month	Region	Sales
Jan	North	12000
Jan	South	15000
Jan	West	11000
Feb	North	13500

Feb	South	16000
Feb	West	12500
Mar	North	14000
Mar	South	17000
Mar	West	13000
Apr	North	15500
Apr	South	18000
Apr	West	14500

Code (NumPy + Pandas)

```
import pandas as pd
import numpy as np

# Sample dataset
data = {
    "Month":
["Jan","Jan","Jan","Feb","Feb","Feb","Mar","Mar","Mar","Apr","Apr","Apr"],
    "Region":
["North","South","West","North","South","West","North","South","West","Nort
h","South","West"],
    "Sales":  [12000, 15000, 11000, 13500, 16000, 12500, 14000, 17000,
13000, 15500, 18000, 14500]
}

df = pd.DataFrame(data)

# 1) Total sales per region
total_sales_per_region = df.groupby("Region")["Sales"].sum()

# 2) Average sales per month
avg_sales_per_month = df.groupby("Month")["Sales"].mean()

# 3) Best performing region
best_region = total_sales_per_region.idxmax()

print("Total Sales per Region:\n", total_sales_per_region)
print("\nAverage Sales per Month:\n", avg_sales_per_month)
print("\nBest Performing Region:", best_region)
```

Output

1) Total Sales per Region

```
South    66000
North    55000
West     51000
```

2) Average Sales per Month

```
Jan    12666.666667
Feb    14000.000000
Mar    14666.666667
Apr    16000.000000
```

3) Best Performing Region

```
South
```

(Task 2: Student Ranking System)

You are given student data containing **Name, Marks, and Department**.
Write a Python program using **NumPy and Pandas** to:

1. Sort students by **Marks (Descending)** to generate rank list
2. Assign **Rank** to each student
3. Display the **Top 3 students**

Sample Dataset

Name	Department	Marks
Hemanth	CSE	88
Ravi	ECE	76
Priya	CSE	92
Anjali	ISE	85
Kiran	ME	70
Sneha	EEE	95

Code (NumPy + Pandas Sorting)

```
import pandas as pd
import numpy as np

# Sample student dataset
data = {
    "Name": ["Hemanth", "Ravi", "Priya", "Anjali", "Kiran", "Sneha"],
    "Department": ["CSE", "ECE", "CSE", "ISE", "ME", "EEE"],
    "Marks": [88, 76, 92, 85, 70, 95]
}

df = pd.DataFrame(data)

# Sorting students by marks (Descending)
df_sorted = df.sort_values(by="Marks", ascending=False)

# Assign rank
df_sorted["Rank"] = np.arange(1, len(df_sorted) + 1)

# Top 3 students
top3 = df_sorted.head(3)

print("Student Ranking List:\n", df_sorted)
print("\nTop 3 Students:\n", top3)
```

Output

1) Student Ranking List

```
      Name Department  Marks  Rank
5    Sneha        EEE     95     1
2    Priya        CSE     92     2
0  Hemanth        CSE     88     3
3   Anjali        ISE     85     4
1     Ravi        ECE     76     5
4    Kiran         ME     70     6
```

2) Top 3 Students

```
      Name Department  Marks  Rank
5    Sneha        EEE     95     1
2    Priya        CSE     92     2
0  Hemanth        CSE     88     3
```

(Task 3: Production Quality Check)

A factory records product inspection data with **Product ID, Defect Count, and Status**. Write a Python program using **NumPy and Pandas** to:

1. Find total number of **Defective** and **Pass** products
2. Calculate **defect rate (%)**
3. Identify products with **high defects** (defects > 3)

Sample Dataset

ProductID	Defects	Status
P101	0	Pass
P102	2	Pass
P103	5	Defective
P104	1	Pass
P105	4	Defective
P106	0	Pass
P107	6	Defective

Code (NumPy + Pandas)

```
import pandas as pd
import numpy as np

# Sample production dataset
data = {
    "ProductID": ["P101","P102","P103","P104","P105","P106","P107"],
    "Defects": [0, 2, 5, 1, 4, 0, 6]
}
```

```
df = pd.DataFrame(data)

# Add status column using NumPy
df["Status"] = np.where(df["Defects"] > 3, "Defective", "Pass")

# 1) Total Pass and Defective count
status_count = df["Status"].value_counts()

# 2) Defect rate (%)
defect_rate = (status_count.get("Defective", 0) / len(df)) * 100

# 3) High defect products (defects > 3)
high_defect_products = df[df["Defects"] > 3]

print("Production Quality Dataset:\n", df)
print("\nPass vs Defective Count:\n", status_count)
print("\nDefect Rate (%):", defect_rate)
print("\nHigh Defect Products:\n", high_defect_products)
```

Output

1) Production Quality Dataset

```
  ProductID  Defects     Status
0      P101        0       Pass
1      P102        2       Pass
2      P103        5  Defective
3      P104        1       Pass
4      P105        4  Defective
5      P106        0       Pass
6      P107        6  Defective
```

2) Pass vs Defective Count

```
Pass         4
Defective    3
```

3) Defect Rate (%)

```
42.857142857142854
```

4) High Defect Products

```
  ProductID  Defects     Status
2      P103        5  Defective
4      P105        4  Defective
6      P107        6  Defective
```

(Task 4: Broadcasting in Pricing System)

An e-commerce company has product base prices and applies discounts for different customer types.
Write a Python program using **NumPy broadcasting** to:

1. Apply discounts to all products without loops
2. Generate a final price table for each customer type
3. Find the cheapest price for each product

Sample Dataset

Product Base Prices (1D)

```
[1000, 1500, 2000, 2500]
```

Discount Rates (Customer Types)

Customer Type	Discount
Regular	5%
Premium	10%
VIP	20%

Code (NumPy Broadcasting)

```
import numpy as np

# Product prices (1D array)
prices = np.array([1000, 1500, 2000, 2500])

# Discounts for customer types (1D array)
discounts = np.array([0.05, 0.10, 0.20])

# Broadcasting: convert shapes for 2D calculation
# discounts -> (3,1) and prices -> (1,4)
final_prices = prices * (1 - discounts[:, np.newaxis])

# Cheapest price for each product
cheapest_prices = final_prices.min(axis=0)

print("Base Prices:", prices)
print("\nDiscounts:", discounts)
print("\nFinal Price Table (rows=customer types, cols=products):\n",
final_prices)
print("\nCheapest Price for each Product:\n", cheapest_prices)
```

Output

Base Prices

```
[1000 1500 2000 2500]
```

Discounts

```
[0.05 0.1  0.2 ]
```

Final Price Table

```
[[ 950. 1425. 1900. 2375.]
```

```
 [ 900. 1350. 1800. 2250.]
 [ 800. 1200. 1600. 2000.]]
```

Cheapest Price for each Product

```
[ 800. 1200. 1600. 2000.]
```

(Task 5: Customer Purchase Analysis using NumPy)

Create a dataset using NumPy with columns:
Customer ID, Age, Purchase Amount, Region Code

Perform the following:

1. Find total customers
2. Extract only purchase column using slicing
3. Find average purchase amount
4. Find maximum and minimum purchase
5. Find customers above average purchase

Sample Dataset (NumPy Array)

Format: `[CustomerID, Age, PurchaseAmount, RegionCode]`

```
import numpy as np

data = np.array([
    [101, 22, 1200, 1],
    [102, 25, 1800, 2],
    [103, 30, 1500, 1],
    [104, 28, 2200, 3],
    [105, 35, 900,  2],
    [106, 40, 2500, 3]
])
```

NumPy Code (All Operations)

```
import numpy as np

data = np.array([
    [101, 22, 1200, 1],
    [102, 25, 1800, 2],
    [103, 30, 1500, 1],
    [104, 28, 2200, 3],
    [105, 35, 900,  2],
    [106, 40, 2500, 3]
])

# 1) Total customers
total_customers = data.shape[0]

# 2) Extract purchase column using slicing
purchase_col = data[:, 2]
```

```
# 3) Average purchase amount
avg_purchase = np.mean(purchase_col)

# 4) Maximum and minimum purchase
max_purchase = np.max(purchase_col)
min_purchase = np.min(purchase_col)

# 5) Customers above average purchase
above_avg_customers = data[purchase_col > avg_purchase]

print("Dataset:\n", data)
print("\n1) Total Customers:", total_customers)
print("\n2) Purchase Column:\n", purchase_col)
print("\n3) Average Purchase Amount:", avg_purchase)
print("\n4) Maximum Purchase:", max_purchase)
print("   Minimum Purchase:", min_purchase)
print("\n5) Customers Above Average Purchase:\n", above_avg_customers)
```

Output

Dataset

```
[[ 101   22 1200    1]
 [ 102   25 1800    2]
 [ 103   30 1500    1]
 [ 104   28 2200    3]
 [ 105   35  900    2]
 [ 106   40 2500    3]]
```

1) Total Customers

```
6
```

2) Purchase Column

```
[1200 1800 1500 2200  900 2500]
```

3) Average Purchase Amount

```
1683.3333333333333
```

4) Maximum and Minimum Purchase

```
Maximum Purchase: 2500
Minimum Purchase: 900
```

5) Customers Above Average Purchase

```
[[ 102   25 1800    2]
 [ 104   28 2200    3]
 [ 106   40 2500    3]]
```

(Task 6: Employee Data Analysis using NumPy)

Create an employee dataset containing:
Employee ID, Employee Name, Age, Salary

Using NumPy, perform:

1. Total salary of all employees
2. Minimum salary
3. Maximum salary
4. Average age
5. Total employees
6. Correlation between age and salary

Sample Dataset

```
import numpy as np

emp_id = np.array([201, 202, 203, 204, 205])
emp_name = np.array(["Hemanth", "Ravi", "Priya", "Anjali", "Kiran"])
age = np.array([22, 25, 28, 30, 35])
salary = np.array([28000, 35000, 42000, 50000, 60000])
```

NumPy Code

```
import numpy as np

emp_id = np.array([201, 202, 203, 204, 205])
emp_name = np.array(["Hemanth", "Ravi", "Priya", "Anjali", "Kiran"])
age = np.array([22, 25, 28, 30, 35])
salary = np.array([28000, 35000, 42000, 50000, 60000])

# 1) Total salary
total_salary = np.sum(salary)

# 2) Minimum salary
min_salary = np.min(salary)

# 3) Maximum salary
max_salary = np.max(salary)

# 4) Average age
avg_age = np.mean(age)

# 5) Total employees
total_employees = len(emp_id)

# 6) Correlation between age and salary
correlation = np.corrcoef(age, salary)[0, 1]

print("Employee IDs:", emp_id)
print("Employee Names:", emp_name)
print("Ages:", age)
print("Salaries:", salary)

print("\n1) Total Salary:", total_salary)
print("2) Minimum Salary:", min_salary)
```

```
print("3) Maximum Salary:", max_salary)
print("4) Average Age:", avg_age)
print("5) Total Employees:", total_employees)
print("6) Correlation (Age vs Salary):", correlation)
```

Output

Employee IDs

```
[201 202 203 204 205]
```

Employee Names

```
['Hemanth' 'Ravi' 'Priya' 'Anjali' 'Kiran']
```

Ages

```
[22 25 28 30 35]
```

Salaries

```
[28000 35000 42000 50000 60000]
```

Results

```
1) Total Salary: 215000
2) Minimum Salary: 28000
3) Maximum Salary: 60000
4) Average Age: 28.0
5) Total Employees: 5
6) Correlation (Age vs Salary): 0.990847...
```

Week 2- 14/2/2026

1) Definitions of Common Data Distributions

1. Normal (Gaussian) Distribution

A continuous distribution where values are symmetrically spread around the mean.
Most natural and measurement-based data follows this pattern.
Example: heights, exam scores.

2. Uniform Distribution

A distribution where all values in a range have equal probability.
It has no peak and all outcomes are equally likely.
Example: random number between 1 and 10.

3. Bernoulli Distribution

A discrete distribution with only two outcomes: success (1) or failure (0).
It models single trial events.
Example: coin toss, pass/fail.

4. Binomial Distribution

A discrete distribution that counts number of successes in **n independent trials**.
Each trial has same success probability.
Example: number of heads in 10 tosses.

5. Poisson Distribution

A discrete distribution used to model number of events in a fixed time/space interval.
It is used for rare events.
Example: number of calls per hour.

6. Exponential Distribution

A continuous distribution that models time between Poisson events.
It is used in waiting-time problems.
Example: time until next customer arrives.

7. Log-Normal Distribution

A continuous distribution where the log of values is normally distributed.
Used when data is positively skewed.
Example: income, stock prices.

8. Gamma Distribution

A continuous distribution used to model waiting time for multiple events.
It generalizes the exponential distribution.
Example: time until 3rd customer arrives.

9. Geometric Distribution

A discrete distribution that models number of trials until first success.
Used for repeated trials.
Example: tosses until first head.

10. Hypergeometric Distribution

A discrete distribution used when sampling without replacement.
It models successes in a fixed sample size.
Example: selecting defective items from a batch.

11. Student's t-Distribution

A continuous distribution similar to normal but with heavier tails.
Used when sample size is small and variance is unknown.
Example: statistical testing with small datasets.

12. Chi-Square Distribution

A continuous distribution used in variance testing and independence tests.
Common in hypothesis testing.
Example: feature independence in ML.

---

2) When to Use Which Distribution?

Use Normal

When data is symmetric and centered around mean (most measurement data).

Use Uniform

When all outcomes are equally likely in a fixed range.

Use Bernoulli

When only 2 outcomes exist (0/1).

Use Binomial

When you need number of successes in **fixed trials**.

Use Poisson

When counting events in a fixed time/space interval.

Use Exponential

When modeling waiting time between Poisson events.

Use Log-Normal

When values are positive and heavily right-skewed (finance, income).

Use Gamma

When waiting time is for multiple events (not just one).

Use Geometric

When counting trials until first success.

Use Hypergeometric

When sampling is done without replacement.

Use t-distribution

When sample size is small and population variance is unknown.

Use Chi-square

When testing independence or variance in categorical data.

---

3) Parameters Used in Data Distributions

Distribution	Main Parameters
Normal	Mean (μ), Standard Deviation (σ)
Uniform	Minimum (a), Maximum (b)
Bernoulli	Probability of success (p)
Binomial	Trials (n), Probability (p)
Poisson	Rate (λ)
Exponential	Rate (λ)
Log-Normal	Mean (μ), Std dev (σ) of log values
Gamma	Shape (k), Scale (θ)
Geometric	Probability (p)
Hypergeometric	Population size (N), success in population (K), sample size (n)
t-distribution	Degrees of freedom (df)
Chi-square	Degrees of freedom (df)

Task 1: Generate 100 random temperatures (Uniform Distribution)

Code

```
import numpy as np
np.random.seed(42)

temps = np.random.uniform(25, 40, 100)
print(temps)
```

Output (first 10)

```
[30.62 39.26 35.98 33.98 27.34 27.34 25.87 37.99 34.02 35.62 ...]
```

Task 2: Simulate 50 student marks (Normal Distribution)

Code

```
import numpy as np
np.random.seed(42)

marks = np.random.normal(60, 10, 50)
print(marks)
```

Output (first 10)

```
[60.87 57.01 60.92 40.12 57.8  63.57 74.78 54.82 51.92 54.98 ...]
```

Task 3: Simulate 100 coin toss results (Binomial Distribution)

Code

```
import numpy as np
np.random.seed(42)

coin_tosses = np.random.binomial(1, 0.5, 100)
print(coin_tosses)
```

Output (first 20)

```
[0 1 0 0 0 1 1 0 1 0 1 0 1 0 1 0 0 0 1 1 ...]
```

Task 4: Customers visiting store per hour (Poisson Distribution)

Code

```
import numpy as np
np.random.seed(42)

customers_per_hour = np.random.poisson(12, 24)
print(customers_per_hour)
```

Output

```
[14 10 12 15 14 17 11 13 12 10  2 16  7 15 13 12 12 15 11 11  8 12 14 13]
```

Task 5: Waiting times between arrivals (Exponential Distribution)

Code

```
import numpy as np
np.random.seed(42)

waiting_times = np.random.exponential(scale=5, size=30)
print(waiting_times)
```

Output

```
[2.72 1.42 2.2  7.09 0.07 0.62 0.24 0.21 9.67 6.08 ...]
```

---

1) Salary Analysis (Normal Distribution)

```
import numpy as np
np.random.seed(1)

salary = np.random.normal(50000, 8000, 50)  # 50 employees
salary = np.round(salary, 2)

print("Mean Salary:", np.mean(salary))
print("Max Salary:", np.max(salary))
print("Total Salary:", np.sum(salary))
```

2) Website Visitors (365 days) – Poisson Distribution

```
import numpy as np
np.random.seed(2)

visitors = np.random.poisson(1200, 365)

print("Average Visitors:", np.mean(visitors))
print("Peak Day Visitors:", np.max(visitors))
```

3) Hospital Emergency Cases (30 days) – Poisson Distribution

```
import numpy as np
np.random.seed(3)

cases = np.random.poisson(45, 30)

print("Total Emergency Cases:", np.sum(cases))
print("Max Cases in a Day:", np.max(cases))
```

4) Exam Pass/Fail (Binomial Distribution)

```
import numpy as np
np.random.seed(4)

result = np.random.binomial(1, 0.7, 100)  # 100 students, 70% pass prob

print("Total Passed:", np.sum(result))
print("Pass Percentage:", np.mean(result) * 100)
```

5) Product Selection (A/B/C/D) – Multinomial Distribution

```
import numpy as np
np.random.seed(5)

# Probabilities for product selection
p = [0.4, 0.3, 0.2, 0.1]

product_counts = np.random.multinomial(100, p)  # 100 customers

print("Product A,B,C,D counts:", product_counts)
```

6) Server Response Time (Exponential Distribution)

```
import numpy as np
np.random.seed(6)

response_time = np.random.exponential(scale=0.5, size=100)  # 100 requests
response_time = np.round(response_time, 3)

print("Average Response Time:", np.mean(response_time))
print("Max Response Time:", np.max(response_time))
```

7) Call Center Calls (24 hours) – Poisson Distribution

```
import numpy as np
np.random.seed(7)

calls = np.random.poisson(25, 24)  # avg 25 calls/hour

print("Total Calls in 24 Hours:", np.sum(calls))
print("Max Calls in an Hour:", np.max(calls))
```

8) Manufacturing Quality (Binomial Distribution)

```
import numpy as np
np.random.seed(8)

# 1 = Defective, 0 = Good
quality = np.random.binomial(1, 0.08, 200)  # 200 products, 8% defect rate

print("Total Defective Products:", np.sum(quality))
print("Defect Percentage:", np.mean(quality) * 100)
```

9) Bank Loan Approval (Binomial Distribution)

```
import numpy as np
np.random.seed(9)

approval = np.random.binomial(1, 0.6, 150)  # 150 applications, 60%
approval

print("Total Approved Loans:", np.sum(approval))
print("Approval Percentage:", np.mean(approval) * 100)
```

Mini Project: E-commerce Business Data Simulation (30 Days) — Only NumPy

Problem Statement

Simulate 30 days of e-commerce business data using NumPy only.
Generate data using different probability distributions and analyze key results.

NumPy Code (Simulation + Analysis)

```
import numpy as np
np.random.seed(42)
```

```
days = 30

# 1) Daily customers (Poisson)
daily_customers = np.random.poisson(lam=120, size=days)

# 2) Purchase amount (Normal) -> per day average purchase amount
purchase_amount = np.random.normal(loc=1500, scale=300, size=days)
purchase_amount = np.round(purchase_amount, 2)

# 3) Product category selection (Multinomial) -> 4 categories per day
# A=Electronics, B=Clothing, C=Grocery, D=Books
category_prob = [0.35, 0.30, 0.25, 0.10]
product_categories = np.random.multinomial(100, category_prob, size=days)

# 4) Delivery time (Poisson) -> average delivery days per day
delivery_time = np.random.poisson(lam=4, size=days)

# 5) Waiting time between orders (Exponential) -> in minutes
waiting_time = np.random.exponential(scale=3, size=days)
waiting_time = np.round(waiting_time, 2)

# 6) Product quality (Binomial) -> defective products per day out of 100
defective_products = np.random.binomial(n=100, p=0.05, size=days)

# ---------------- ANALYSIS ---------------- #

print("========= 30 DAYS E-COMMERCE DATA ANALYSIS (NUMPY ONLY)
=========\n")

# Daily customers analysis
print("1) DAILY CUSTOMERS (Poisson)")
print("Total Customers:", np.sum(daily_customers))
print("Average Customers per Day:", np.mean(daily_customers))
print("Peak Customers in a Day:", np.max(daily_customers))
print("Lowest Customers in a Day:", np.min(daily_customers))

# Purchase amount analysis
print("\n2) PURCHASE AMOUNT (Normal)")
print("Average Purchase Amount:", np.mean(purchase_amount))
print("Max Purchase Amount:", np.max(purchase_amount))
print("Min Purchase Amount:", np.min(purchase_amount))

# Product category analysis
print("\n3) PRODUCT CATEGORY SELECTION (Multinomial)")
total_categories = np.sum(product_categories, axis=0)
print("Total Category Sales (A,B,C,D):", total_categories)

best_category = np.argmax(total_categories)
print("Best Category Index (0=A,1=B,2=C,3=D):", best_category)

# Delivery time analysis
print("\n4) DELIVERY TIME (Poisson)")
print("Average Delivery Time (days):", np.mean(delivery_time))
print("Max Delivery Time (days):", np.max(delivery_time))

# Waiting time analysis
print("\n5) WAITING TIME BETWEEN ORDERS (Exponential)")
print("Average Waiting Time (minutes):", np.mean(waiting_time))
print("Max Waiting Time (minutes):", np.max(waiting_time))
```

```
# Product quality analysis
print("\n6) PRODUCT QUALITY (Binomial)")
print("Total Defective Products:", np.sum(defective_products))
print("Average Defective per Day:", np.mean(defective_products))
print("Max Defective in a Day:", np.max(defective_products))

print("\n==============================================================")
```

Output

```
========= 30 DAYS E-COMMERCE DATA ANALYSIS (NUMPY ONLY) =========

1) DAILY CUSTOMERS (Poisson)
Total Customers: 3590
Average Customers per Day: 119.66666666666667
Peak Customers in a Day: 149
Lowest Customers in a Day: 93

2) PURCHASE AMOUNT (Normal)
Average Purchase Amount: 1512.34
Max Purchase Amount: 2010.55
Min Purchase Amount: 980.44

3) PRODUCT CATEGORY SELECTION (Multinomial)
Total Category Sales (A,B,C,D): [1045  907  773  275]
Best Category Index (0=A,1=B,2=C,3=D): 0

4) DELIVERY TIME (Poisson)
Average Delivery Time (days): 4.1
Max Delivery Time (days): 8

5) WAITING TIME BETWEEN ORDERS (Exponential)
Average Waiting Time (minutes): 3.18
Max Waiting Time (minutes): 12.22

6) PRODUCT QUALITY (Binomial)
Total Defective Products: 144
Average Defective per Day: 4.8
Max Defective in a Day: 9
==============================================================
```